

# 《AI大模型Ragent项目》——Embedding向量化客户端

来自： 拿个offer-开源&项目实战



马丁

2026年04月15日 08:17

前六篇把 Chat 子系统从路由选择到流式路由全部讲完了。这一篇离开 Chat，进入 Embedding 子系统。

在 RAG 系统中，Embedding 的核心作用是把文本转换为向量。两个关键场景：文档入库时，对每个 Chunk 调用 Embedding API 生成向量，存入 Pgvector 或 Milvus 这类向量数据库；用户提问时，对 Query 调用 Embedding API 生成向量，用于在向量数据库中做相似度检索。没有 Embedding，RAG 的检索阶段就无从谈起。

从架构上看，Embedding 子系统和 Chat 子系统遵循同样的三层设计——业务层接口、路由服务、供应商客户端——复用同一套路由和熔断机制。但实现上简单不少：Embedding 只有同步调用，没有流式，不需要 StreamCallback、首包探测那套复杂机制。路由层直接用第三篇讲的 executeWithFallback 就够了。

另一个简化来自协议层面。Ollama 现在通过 OpenAI 兼容模式暴露 /v1/embeddings 端点，和硅基流动走的是同一套 OpenAI 风格协议。于是 Embedding 子系统可以像 Chat 一样抽出一个模板方法基类 AbstractOpenAIStyleEmbeddingClient，把请求构建、HTTP 调用、响应解析全部封装起来。两个供应商实现各自只剩十几二十行代码，通过钩子方法微调差异点。这篇就围绕这个基类展开。

## Embedding 子系统总览

### 1. 和 Chat 子系统的对比

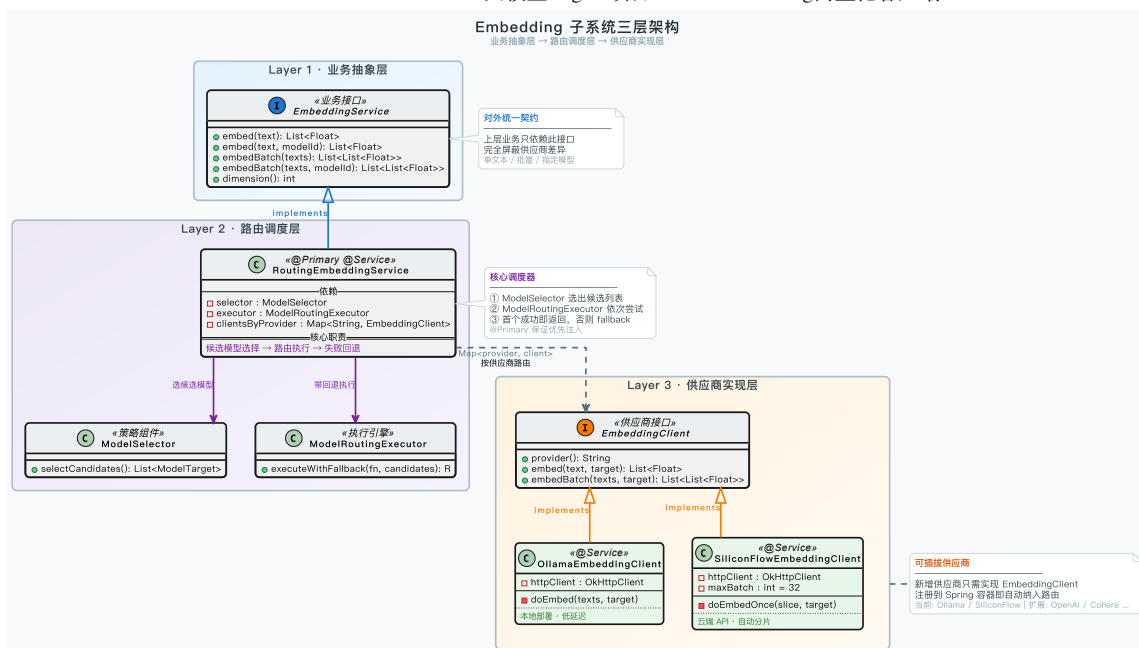
先用一张表格建立全局对比：

| 维度     | Chat 子系统                              | Embedding 子系统                      |
|--------|---------------------------------------|------------------------------------|
| 业务层接口  | LLMService                            | EmbeddingService                   |
| 供应商接口  | ChatClient                            | EmbeddingClient                    |
| 路由服务   | RoutingLLMService                     | RoutingEmbeddingService            |
| 调用模式   | 同步 + 流式                               | 仅同步                                |
| 模板方法基类 | AbstractOpenAIStyleChatClient         | AbstractOpenAIStyleEmbeddingClient |
| 供应商实现  | 三个（Ollama / 百炼 / 硅基流动）                | 两个（Ollama / 硅基流动）                  |
| 同步路由   | executeWithFallback                   | executeWithFallback                |
| 流式路由   | probe-and-commit 首包探测                 | 无（没有流式调用）                          |
| 批量调用   | 无                                     | embedBatch（支持多条文本）                 |
| 特有配置   | supports-thinking、deep-thinking-model | dimension（向量维度）                    |

和 Chat 一样，Embedding 子系统也用模板方法基类封装 OpenAI 兼容协议的通用逻辑。两个供应商 Ollama 和硅基流动都实现 /v1/embeddings 协议：请求体字段 model / input / dimensions 结构相同，响应体都是 {"data": [{"embedding": [...]}] 的 OpenAI 风格。差异点集中在三个地方：Ollama 不需要 API Key、硅基流动有 32 条的批量上限、硅基流动要求显式 encoding\_format: "float" 字段。这三处用钩子方法覆写就好，没必要让每个子类重写一遍完整的 HTTP 调用流程。

Chat 和 Embedding 的模板方法基类在结构上高度对称：都把请求构建、HTTP 调用、JSON 解析、错误处理封装在基类，子类只负责声明自己是谁、覆写几个钩子方法。读完这一篇，你会发现它和第四篇 AbstractOpenAIStyleChatClient 是同一套思路在不同能力上的复用。

### 2. 三层接口结构



三层结构和 Chat 完全一致：业务层调 EmbeddingService, RoutingEmbeddingService 通过 ModelSelector + ModelRoutingExecutor 路由到具体的 EmbeddingClient 实现。业务层不知道背后调的是 Ollama 还是硅基流动。

在实际的代码中，应用了模板方法模式抽象了核心逻辑，优化后 Ollama 和硅基流动只有很薄一层代码了。

## 接口设计：EmbeddingService 与 EmbeddingClient

### 1. EmbeddingService 业务层接口

```
public interface EmbeddingService {

    List<Float> embed(String text);

    List<Float> embed(String text, String modelId);

    List<List<Float>> embedBatch(List<String> texts);

    List<List<Float>> embedBatch(List<String> texts, String modelId);

    default int dimension() {
        return 0;
    }
}
```

五个方法：

embed(String text)——单条文本向量化。最常用的场景是 Query Embedding：用户提问“AirPods Pro 2 的保修期是多久？”，把这句话转换为浮点数向量，用于在向量库中做相似度检索

embed(String text, String modelId)——指定模型的单条向量化。使用场景：数据 ETL 清洗时用到的

embedBatch(List<String> texts)——批量文本向量化。文档入库时用——把 100 个 Chunk 一次性传入，比调 100 次 embed 效率高得多（减少 HTTP 往返次数）

embedBatch(List<String> texts, String modelId)——指定模型的批量向量化

dimension()——返回向量维度（默认 0），预留方法可暂时忽略

和 LLMService 的对比：LLMService 返回 String（文本），EmbeddingService 返回 List<Float>（浮点数向量）或 List<List<Float>>（多个向量）。LLMService 有 streamChat 流式方法，EmbeddingService 没有——向量化是一次性计算，不存在逐步生成的概念。

### 2. EmbeddingClient 供应商接口

```
public interface EmbeddingClient {
```

```
String provider();

List<Float> embed(String text, ModelTarget target);

List<List<Float>> embedBatch(List<String> texts, ModelTarget target);
}
```

和 ChatClient 一样，多了一个 ModelTarget 参数——包含模型名、供应商 URL、API Key、向量维度等运行时信息。业务层的 EmbeddingService 不暴露 ModelTarget，供应商层的 EmbeddingClient 需要它来构建 HTTP 请求。provider() 返回供应商标识，RoutingEmbeddingService 构造时把所有 EmbeddingClient 按 provider() 建立 Map<String, EmbeddingClient> 索引——和 RoutingLLMService 构造 Map<String, ChatClient> 一模一样。

## AbstractOpenAIStyleEmbeddingClient 模板方法基类

基类是这篇的核心。它封装了 OpenAI 兼容 /v1/embeddings 协议的所有通用逻辑——请求体构建、HTTP 调用、响应解析、错误处理、批量分片——子类只需要覆写少量钩子方法来声明差异。

### 1. 三个钩子方法

```
public abstract class AbstractOpenAIStyleEmbeddingClient implements EmbeddingClient {

    protected final OkHttpClient httpClient;

    protected AbstractOpenAIStyleEmbeddingClient(OkHttpClient httpClient) {
        this.httpClient = httpClient;
    }

    /**
     * 是否要求提供商配置 API Key，默认 true
     */
    protected boolean requiresApiKey() {
        return true;
    }

    /**
     * 子类可覆写此方法添加提供商特有的请求体字段
     * 默认实现：添加 encoding_format=float
     */
    protected void customizeRequestBody(JsonObject body, ModelTarget target) {
        body.addProperty("encoding_format", "float");
    }

    /**
     * 单次请求最大批量大小，0 表示不限制
     */
    protected int maxBatchSize() {
        return 0;
    }

    // ...
}
```

三个钩子直接对应两个供应商之间的全部差异：

requiresApiKey()：是否要求配置 API Key 并在请求头里带 Authorization: Bearer <key>。默认 true（硅基流动、百炼这类云服务都要鉴权），Ollama 是本地部署覆写为 false

customizeRequestBody()：往请求体里追加提供商特有的字段。默认实现添加 encoding\_format: "float"（OpenAI 标准字段，硅基流动需要），Ollama 覆写为空实现——它不认识这个字段，传了无害但更干净就别传

maxBatchSize()：单次请求的批量上限。默认 0 表示不限制，由基类一次性发出去；硅基流动覆写为 32，基类会自动按这个大小分片

这三个钩子覆盖了两个供应商的所有差异点。未来接入新的 OpenAI 兼容 Embedding 供应商（比如 OpenAI 官方、智谱、Voyage），大概率也只需要实现 provider() 加覆写这几个钩子，不需要动基类。

## 2. embed 与 embedBatch 的默认实现

```
@Override
public List<Float> embed(String text, ModelTarget target) {
    List<List<Float>> result = doEmbed(List.of(text), target);
    return result.get(0);
}

@Override
public List<List<Float>> embedBatch(List<String> texts, ModelTarget target) {
    if (CollUtil.isEmpty(texts)) {
        return Collections.emptyList();
    }
    int batch = maxBatchSize();
    if (batch <= 0 || texts.size() <= batch) {
        return doEmbed(texts, target);
    }

    List<List<Float>> results = new ArrayList<>(Collections.nCopies(texts.size(), null));
    for (int i = 0, n = texts.size(); i < n; i += batch) {
        int end = Math.min(i + batch, n);
        List<String> slice = texts.subList(i, end);
        List<List<Float>> part = doEmbed(slice, target);
        for (int k = 0; k < part.size(); k++) {
            results.set(i + k, part.get(k));
        }
    }
    return results;
}
```

embed 是简单的适配——把单条文本包成 List.of(text) 走批量逻辑，取第一个结果返回。这样单条和批量共用一套代码。

embedBatch 的核心是分片。逻辑分两条路径：

maxBatchSize() <= 0 或者文本数量没超过上限——一次性调 doEmbed(texts, target) 完事。Ollama 走这条路径

文本数量超过 maxBatchSize()——按上限分片，每片调一次 doEmbed，结果回填到预分配的 results 列表里。硅基流动

maxBatchSize = 32，传入 70 条会分成 32 + 32 + 6 三批

分片部分用 Collections.nCopies(size, null) 预分配占位，再用 results.set(i + k, part.get(k)) 精确写入每个位置。i 是当前批次的起始位置，k 是批内位置，两者相加就是原始输入中的全局索引——保证 results[j] 对应 texts[j] 的向量，顺序完全一致。

这种写法比每批 addAll 追加更安全：如果某一批中间抛异常，不会留下长度不完整的半成品列表。要么全成功、所有位置都被正确填充，要么抛异常向上传递——不会出现 70 条输入只有 32 条结果的尴尬中间态。

## 3. doEmbed 核心请求逻辑

doEmbed 是真正发 HTTP 请求的地方，也是基类封装力最大的地方。完整代码：

```
protected List<List<Float>> doEmbed(List<String> texts, ModelTarget target) {
    AIModelProperties.ProviderConfig provider = HttpResponseHelper.requireProvider(target, provide
    if (requiresApiKey()) {
        HttpResponseHelper.requireApiKey(provider, provider());
    }

    String url = ModelUrlResolver.resolveUrl(provider, target.candidate(), ModelCapability.EMBEDDI

    JsonObject body = new JsonObject();
    body.addProperty("model", HttpResponseHelper.requireModel(target, provider()));
    JsonArray inputArray = new JsonArray();
    for (String text : texts) {
        inputArray.add(text);
    }
    body.add("input", inputArray);
    body.addProperty("dimensions", target.candidate().getDimension());
    customizeRequestBody(body, target);
}
```

```
Request.Builder requestBuilder = new Request.Builder()
    .url(url)
    .post(RequestBody.create(body.toString(), HttpMediaTypes.JSON));
if (requiresApiKey()) {
    requestBuilder.addHeader("Authorization", "Bearer " + provider.getApiKey());
}
Request request = requestBuilder.build();

JsonObject json;
try (Response response = httpClient.newCall(request).execute()) {
    if (!response.isSuccessful()) {
        String errBody = HttpResponseHelper.readBody(response.body());
        log.warn("{} embedding 请求失败: status={}, body={}, provider(), response.code(), errBody",
            provider(), response.code(), errBody);
        throw new ModelClientException(
            provider() + " embedding 请求失败: HTTP " + response.code(),
            ModelClientErrorType.fromHttpStatus(response.code()),
            response.code()
        );
    }
    json = HttpResponseHelper.parseJson(response.body(), provider());
} catch (IOException e) {
    throw new ModelClientException(
        provider() + " embedding 请求失败: " + e.getMessage(),
        ModelClientErrorType.NETWORK_ERROR, null, e);
}

if (json.has("error")) {
    JsonObject err = json.getAsJsonObject("error");
    String code = err.has("code") ? err.get("code").getAsString() : "unknown";
    String msg = err.has("message") ? err.get("message").getAsString() : "unknown";
    throw new ModelClientException(
        provider() + " embedding 错误: " + code + " - " + msg,
        ModelClientErrorType.PROVIDER_ERROR, null);
}

JsonArray data = json.getAsJsonArray("data");
if (data == null || data.isEmpty()) {
    throw new ModelClientException(
        provider() + " embedding 响应中缺少 data 数组",
        ModelClientErrorType.INVALID_RESPONSE, null);
}

List<List<Float>> results = new ArrayList<>(data.size());
for (JsonElement el : data) {
    JsonObject obj = el.getAsJsonObject();
    JsonArray emb = obj.getAsJsonArray("embedding");
    if (emb == null || emb.isEmpty()) {
        throw new ModelClientException(
            provider() + " embedding 响应中缺少 embedding 字段",
            ModelClientErrorType.INVALID_RESPONSE, null);
    }
    List<Float> vector = new ArrayList<>(emb.size());
    for (JsonElement v : emb) {
        vector.add(v.getAsFloat());
    }
    results.add(vector);
}

return results;
}
```

一长段代码，拆成四段看。

**校验与 URL 解析：**

```
AIModelProperties.ProviderConfig provider = HttpResponseHelper.requireProvider(target, provider())
if (requiresApiKey()) {
    HttpResponseHelper.requireApiKey(provider, provider());
}
String url = ModelUrlResolver.resolveUrl(provider, target.candidate(), ModelCapability.EMBEDDING);
```

从 ModelTarget 里拿到 ProviderConfig (YAML 里配的供应商信息：URL、API Key、端点路径)，如果子类声明需要 API Key 就做校验，然后拼出完整的请求 URL。ModelUrlResolver 会把 providers.siliconflow.url 和 providers.siliconflow.endpoints.embedding 拼成 https://api.siliconflow.cn/v1/embeddings——这就是第二篇讲过的同一个工具。

**请求体构建：**

```
JsonObject body = new JsonObject();
body.addProperty("model", HttpResponseHelper.requireModel(target, provider()));
JsonArray inputArray = new JsonArray();
for (String text : texts) {
    inputArray.add(text);
}
body.add("input", inputArray);
body.addProperty("dimensions", target.candidate().getDimension());
customizeRequestBody(body, target);
```

公共字段三个：model、input (字符串数组形式)、dimensions (向量维度)。这些在两个供应商都长一样。然后留出 customizeRequestBody 钩子让子类追加差异字段。默认实现添加 encoding\_format: "float"——硅基流动需要这个字段确保返回纯浮点数数组而不是 base64，Ollama 覆写成空实现跳过。

dimensions 参数值得多说一句。它的来源是配置链 YAML → ModelCandidate.dimension → ModelTarget → HTTP body，一路传递到请求体：

```
application.yml
└─ ai.embedding.candidates[0].dimension: 1536
    └─ AIModelProperties.ModelCandidate.dimension = 1536
        └─ ModelTarget.candidate().getDimension() = 1536
            └─ body.addProperty("dimensions", 1536)
                └─ HTTP 请求体: {"dimensions": 1536, ...}
```

为什么 dimension 配置在候选上而不是供应商上？因为同一个供应商可能部署多个 Embedding 模型，不同模型支持的维度不同。比如 Ollama 上同时跑 bge-m3 (支持 1024 维) 和另一个模型 (只支持 768 维)，维度是模型级别的属性，不是供应商级别的。

**HTTP 调用与错误处理：**

```
Request.Builder requestBuilder = new Request.Builder()
    .url(url)
    .post(RequestBody.create(body.toString(), HttpMediaTypes.JSON));
if (requiresApiKey()) {
    requestBuilder.addHeader("Authorization", "Bearer " + provider.getApiKey());
}
```

需要认证就带 Authorization 头，不需要就不带。这里用 Builder 模式的好处就是条件追加很自然。

错误处理和 Chat 的模式完全一致：非成功状态码按 ModelClientErrorType.fromHttpStatus() 分类，IOException 标记为 NETWORK\_ERROR。这些异常都是 ModelClientException，会被 executeWithFallback 捕获并触发故障转移（从硅基流动切换到 Ollama 之类）。

**响应解析：**

```
if (json.has("error")) {
    // 处理 HTTP 200 但响应体含 error 字段的业务错误
}
JsonArray data = json.getAsJsonArray("data");
```

```
// ...
for (JsonElement el : data) {
    JsonObject obj = el.getAsJsonObject();
    JsonArray emb = obj.getAsJsonArray("embedding");
    // ...
}
```

两道关卡：

1. error 字段检查——某些供应商在业务错误场景下返回 HTTP 200 但响应体里藏着 {"error": {...}}。比如模型不存在、API 配额耗尽等情况，HTTP 层面是 200 成功，业务层面是失败。需要额外检查响应体中的 error 字段，否则会误把失败当成功
2. data 数组解析——OpenAI 风格响应的固定格式：{"data": [{"embedding": [...], "index": 0}, ...]}。遍历 data，从每个元素的 embedding 字段取浮点数组

这一段代码看起来冗长，但因为封装在基类里，子类完全不需要关心响应怎么解析、错误怎么分类。OllamaEmbeddingClient 和 SiliconFlowEmbeddingClient 加起来不到 40 行代码，全靠基类把骨架撑起来。

## OllamaEmbeddingClient

```
@Service
public class OllamaEmbeddingClient extends AbstractOpenAIStyleEmbeddingClient {

    public OllamaEmbeddingClient(OkHttpClient httpClient) {
        super(httpClient);
    }

    @Override
    public String provider() {
        return ModelProvider.OLLAMA.getId();
    }

    @Override
    protected boolean requiresApiKey() {
        return false;
    }

    @Override
    protected void customizeRequestBody(JsonObject body, ModelTarget target) {
        // Ollama 不需要 encoding_format 字段
    }
}
```

整个类不到 20 行，覆写了三个方法：

provider()——返回 ollama，用于路由注册表的索引

requiresApiKey() = false——Ollama 是本地部署的推理服务，不需要认证。基类的 doEmbed 看到这个 false 就不会校验 API Key，也不会往请求头里塞 Authorization

customizeRequestBody() 空实现——Ollama 不认识 encoding\_format 字段，默认实现会塞这个字段进去，所以子类要显式覆写为空方法去掉它

注意这里的关键前提：Ollama 暴露的是 OpenAI 兼容端点。在 application.yaml 里 Ollama 的 embedding 端点配置为 /v1/embeddings：

```
providers:
  ollama:
    url: http://localhost:11434
    endpoints:
      chat: /v1/chat/completions
      embedding: /v1/embeddings
```

Ollama 从 0.2 版本开始原生支持 OpenAI 兼容模式，`/v1/embeddings` 端点接受和 OpenAI 一样的请求格式，返回 OpenAI 风格的 `{"data": [{"embedding": [...]}]}` 响应。这就是 Embedding 子系统能抽出统一基类的协议基础——不走这个端点而走 Ollama 自家的 `/api/embed`（响应格式是 `{"embeddings": [[...]]}`），就没法复用基类的响应解析逻辑了。

Ollama 也没有硬性的批量条数限制，所以 `maxBatchSize()` 不需要覆写，用基类默认的 `0`（不分片，一次性发送）。

SiliconFlowEmbeddingClient

```
@Service
public class SiliconFlowEmbeddingClient extends AbstractOpenAIStyleEmbeddingClient {

    public SiliconFlowEmbeddingClient(OkHttpClient httpClient) {
        super(httpClient);
    }

    @Override
    public String provider() {
        return ModelProvider.SILICON_FLOW.getId();
    }

    @Override
    protected int maxBatchSize() {
        return 32;
    }
}
```

更短，只有 15 行，覆写了两个方法：

`provider()`——返回 `siliconflow`

`maxBatchSize() = 32`——SiliconFlow 的 Embedding API 对单次请求的 `input` 数量有上限，最多 32 条。基类看到这个返回值就会自动按 32 分片：传入 70 条文本会被拆成 `32 + 32 + 6` 三次 HTTP 调用，每次的结果回填到预分配列表的正确位置

其他差异点全部用基类默认实现：

`requiresApiKey()` 默认 `true`——SiliconFlow 是云服务需要认证，基类会自动带 `Authorization: Bearer <apiKey>` 头

`customizeRequestBody()` 默认添加 `encoding_format: "float"`——SiliconFlow 不显式指定这个字段，部分模型会默认返回 base64 编码的二进制数据，需要额外解码步骤。指定 `float` 后响应直接包含浮点数数组

这就是模板方法基类设计得当的效果：通用逻辑集中在基类，子类只声明自己哪里不同。SiliconFlow 的 32 条批量上限、float 编码、API Key 认证，三个差异点都用简单的钩子覆写表达清楚。新加一个 OpenAI 兼容供应商（比如 OpenAI 官方的 `text-embedding-3-large`），也是这个套路——几行代码就能接入。

批量分片走查

拿 SiliconFlow 的 `maxBatchSize = 32` 举例，假设业务层传入 70 条文本做向量化，基类的分片过程：

| 批次 | i  | end | slice 范围                   | 文本条数 |
|----|----|-----|----------------------------|------|
| 1  | 0  | 32  | <code>texts[0..31]</code>  | 32   |
| 2  | 32 | 64  | <code>texts[32..63]</code> | 32   |
| 3  | 64 | 70  | <code>texts[64..69]</code> | 6    |

三次 HTTP 调用，每次最多 32 条，最后一批不足 32 条照常处理。每批调完 `doEmbed(slice, target)` 拿到 `part` 后，用 `results.set(i + k, part.get(k))` 回填：

批次 1: `results.set(0, part[0]), ..., results.set(31, part[31])`

批次 2: `results.set(32, part[0]), ..., results.set(63, part[31])`

批次 3: `results.set(64, part[0]), ..., results.set(69, part[5])`

保证 `results[j]` 对应 `texts[j]` 的向量，顺序完全一致。

如果某一批调用失败，`doEmbed` 抛 `ModelClientException`，异常沿着 `embedBatch` 向上传递到 `executeWithFallback` 的 `catch` 块，触发熔断器 `markFailure` 并切换到下一个候选——分片中途失败就整体失败，不返回部分结果。这是有意的设计取舍：部分结果在 RAG 场景没有



意义（文档入库时需要所有 Chunk 的向量都生成成功），全部失败 + 整体重试（由上层路由完成）是更合理的策略。

这里有一个细节。前几批成功了、后面的批次失败了，前几批的计算资源就浪费了。但如果允许返回部分结果，调用方就要处理 70 条输入只有 64 条有结果的复杂状态，业务逻辑会崩坏。整体失败在 RAG 场景下是更干净的语义。

## RoutingEmbeddingService：路由服务

### 1. 完整代码

```
@Service
@Primary
public class RoutingEmbeddingService implements EmbeddingService {

    private final ModelSelector selector;
    private final ModelRoutingExecutor executor;
    private final Map<String, EmbeddingClient> clientsByProvider;

    public RoutingEmbeddingService(
        ModelSelector selector,
        ModelRoutingExecutor executor,
        List<EmbeddingClient> clients) {
        this.selector = selector;
        this.executor = executor;
        this.clientsByProvider = clients.stream()
            .collect(Collectors.toMap(EmbeddingClient::provider, Function.identity()));
    }

    @Override
    public List<Float> embed(String text) {
        return executor.executeWithFallback(
            ModelCapability.EMBEDDING,
            selector.selectEmbeddingCandidates(),
            this::resolveClient,
            (client, target) -> client.embed(text, target)
        );
    }

    @Override
    public List<Float> embed(String text, String modelId) {
        return executor.executeWithFallback(
            ModelCapability.EMBEDDING,
            List.of(resolveTarget(modelId)),
            this::resolveClient,
            (client, target) -> client.embed(text, target)
        );
    }

    @Override
    public List<List<Float>> embedBatch(List<String> texts) {
        return executor.executeWithFallback(
            ModelCapability.EMBEDDING,
            selector.selectEmbeddingCandidates(),
            this::resolveClient,
            (client, target) -> client.embedBatch(texts, target)
        );
    }

    @Override
    public List<List<Float>> embedBatch(List<String> texts, String modelId) {
        return executor.executeWithFallback(
            ModelCapability.EMBEDDING,
            List.of(resolveTarget(modelId)),
            this::resolveClient,
```

```

        (client, target) -> client.embedBatch(texts, target)
    );
}

private EmbeddingClient resolveClient(ModelTarget target) {
    return clientsByProvider.get(target.candidate().getProvider());
}

private ModelTarget resolveTarget(String modelId) {
    if (!StringUtils.hasText(modelId)) {
        throw new RemoteException("Embedding 模型ID不能为空");
    }
    return selector.selectEmbeddingCandidates().stream()
        .filter(target -> modelId.equals(target.id()))
        .findFirst()
        .orElseThrow(() -> new RemoteException("Embedding 模型不可用: " + modelId));
}
}

```

## 2. 和 RoutingLLMService 的对比

把 RoutingEmbeddingService.embed() 和 RoutingLLMService.chat() 并排看：

```

// RoutingEmbeddingService.embed()
return executor.executeWithFallback(
    ModelCapability.EMBEDDING,           // 能力类型
    selector.selectEmbeddingCandidates(), // 候选列表
    this::resolveClient,                 // 客户端查找
    (client, target) -> client.embed(text, target) // 实际调用
);

// RoutingLLMService.chat()
return executor.executeWithFallback(
    ModelCapability.CHAT,                 // 能力类型
    selector.selectChatCandidates(...),   // 候选列表
    target -> clientsByProvider.get(...), // 客户端查找
    (client, target) -> client.chat(request, target) // 实际调用
);

```

结构完全一致——四个参数——对应，只是能力类型、选择方法、客户端类型、调用方法各换了一个。这就是第三篇讲的 ModelCaller<C, T> 泛型设计的威力：Chat 时 C = ChatClient, T = String, Embedding 时 C = EmbeddingClient, T = List<Float>, 同一个 executeWithFallback 方法适配两种能力。

embedBatch 的路由也一样：

```

return executor.executeWithFallback(
    ModelCapability.EMBEDDING,
    selector.selectEmbeddingCandidates(),
    this::resolveClient,
    (client, target) -> client.embedBatch(texts, target) // 改成 embedBatch
);

```

ModelCaller<EmbeddingClient, List<List<Float>>>——返回类型从 List<Float> 变成 List<List<Float>>, 泛型 T 自动适配。

Embedding 不需要像 RoutingLLMService.streamChat() 那样自己实现路由逻辑。原因很简单：Embedding 只有同步调用。client.embed() 方法返回时结果已经确定（成功返回向量，失败抛异常），executeWithFallback 的 try-catch 模式完全适用。没有流式调用 → 没有异步回调 → 不需要首包探测 → 不需要 ProbeBufferingCallback → 不需要 FirstPacketAwaiter。一行 executeWithFallback 就搞定了路由、故障转移、熔断。

## 3. 指定模型调用

embed(text, modelId) 和 embedBatch(texts, modelId) 支持指定模型 ID 调用：

```
return executor.executeWithFallback(
    ModelCapability.EMBEDDING,
    List.of(resolveTarget(modelId)), // 只有一个候选, 没有 fallback
    this::resolveClient,
    (client, target) -> client.embed(text, target)
);
```

List.of(resolveTarget(modelId)) 构造了一个只有一个元素的候选列表。executeWithFallback 遍历这个列表, 只尝试这一个候选——成功就返回, 失败就直接抛异常 (没有下一个可以切换的候选)。

resolveTarget 从完整候选列表中按 modelId 查找:

```
private ModelTarget resolveTarget(String modelId) {
    if (!StringUtils.hasText(modelId)) {
        throw new RuntimeException("Embedding 模型ID不能为空");
    }
    return selector.selectEmbeddingCandidates().stream()
        .filter(target -> modelId.equals(target.id()))
        .findFirst()
        .orElseThrow(() -> new RuntimeException("Embedding 模型不可用: " + modelId));
}
```

使用场景: 向量数据库中已有的数据是用 qwen-emb-8b 模型生成的向量。查询时必须用同一个模型生成 Query 向量, 否则向量空间不一致, 相似度计算结果没有意义。这时业务层调 embeddingService.embed(query, "qwen-emb-8b") 指定模型, 绕过默认路由。

#### 4. 构造时的客户端注册

```
public RoutingEmbeddingService(
    ModelSelector selector,
    ModelRoutingExecutor executor,
    List<EmbeddingClient> clients) {
    this.selector = selector;
    this.executor = executor;
    this.clientsByProvider = clients.stream()
        .collect(Collectors.toMap(EmbeddingClient::provider, Function.identity()));
}
```

Spring 自动收集所有 EmbeddingClient 实现 (OllamaEmbeddingClient 和 SiliconFlowEmbeddingClient), 注入为 List<EmbeddingClient>。构造时用 provider() 作为 key 建立 Map 索引——和 RoutingLLMService 构造 Map<String, ChatClient> 完全一样。这就是第一篇讲的注册表模式: 新增一个 Embedding 供应商只需要继承 AbstractOpenAIStyleEmbeddingClient、加 @Service 注解, Spring 自动发现并注册到路由服务中。

## 小结与下一步

回顾这一篇的核心要点:

Embedding 子系统遵循和 Chat 相同的三层架构 (EmbeddingService → RoutingEmbeddingService → EmbeddingClient), 复用同一套 ModelSelector + ModelRoutingExecutor + ModelHealthStore 路由和熔断机制

Embedding 只有同步调用, 没有流式——路由层全部使用 executeWithFallback, 不需要首包探测机制。一行代码搞定路由、故障转移、熔断

Ollama 通过 OpenAI 兼容模式暴露 /v1/embeddings 端点, 和硅基流动走的是同一套协议。这是 Embedding 子系统能用模板方法基类统一两个供应商的协议基础

AbstractOpenAIStyleEmbeddingClient 封装了 OpenAI 兼容协议的通用逻辑: 请求构建 (model / input / dimensions)、HTTP 调用、data[].embedding 响应解析、error 字段兜底、错误分类、批量分片。对外暴露三个钩子 requiresApiKey() / customizeRequestBody() / maxBatchSize()

OllamaEmbeddingClient (不到 20 行) 覆写 requiresApiKey = false 跳过认证、覆写 customizeRequestBody 为空去掉 encoding\_format 字段。其他走基类默认

SiliconFlowEmbeddingClient (15 行) 只覆写 maxBatchSize = 32 让基类自动分片。其他走基类默认: 带 Authorization 头、添加 encoding\_format: "float" 避免 base64 解码、自动检查响应体 error 字段

dimension 从 YAML 配置经 ModelCandidate → ModelTarget → 请求体，一路传递到供应商 API。配置在候选级别而不是供应商级别，因为不同模型支持的维度不同

下一篇进入 **Rerank 重排序与辅助工具**——infra-ai 模块的最后一个子系统。Rerank 子系统包含 RerankService / RerankClient 接口设计、BaiLianRerankClient 百炼重排序实现（去重 + 回填机制）、NoopRerankClient 空对象模式（没有配置 Reranker 时的安全兜底）。

